

Seminar Presentation

Extracting Syntactic Patterns from Databases

Roland Bernard

December 2025

Andrew Ilyas et al. (2018). "Extracting syntactical patterns from databases". In: 2018 IEEE 34th International Conference on Data Engineering (ICDE). IEEE, pp. 41–52

Introduction

- ▶ Goal: Given a database column of string, infer its syntactic structure.
- ▶ This is strictly single-column analysis, no relational context needed.
- ▶ Problem formulation:
 - Given a set of example strings S ,
 - produce a compact description D of a language that contains them. $S \subseteq \mathcal{L}(D)$
 - This description should generalize beyond the examples.

CHEMBL	Date	ZIP Code
CHEMBL102034	12/27/2016	2111
CHEMBL102036	12/12/2015	2127
CHEMBL102037	01/24/2003	2110
CHEMBL102038	02/15/87	121091402
CHEMBL102040	10/10/86	2215



CHEMBL[0-9][0-9][0-9][0-9][0-9][0-9]

(0|1)[0-9]/(0|1|2|3)[0-9]/([0-9][0-9])?[0-9][0-9]

[0-9][0-9][0-9][0-9]

Motivation

- ▶ Structured summaries of columns are:
 - More compact than full datasets.
 - Easier to compare, search, and interpret.

Proposed Applications:

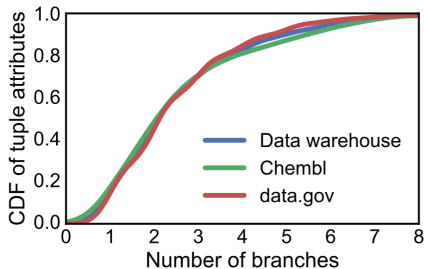
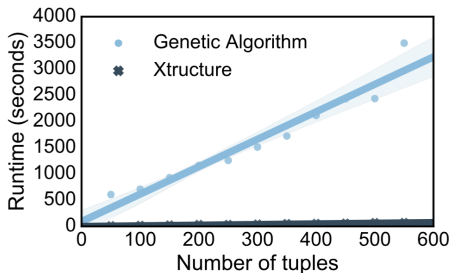
- ▶ Automatic label assignment:
 - Match a column's pattern to a library of known regex-semantic-label pairs.
- ▶ Attribute similarity comparison:
 - Identify columns with compatible syntactic formats (e.g., potential join keys).
- ▶ Syntax-based outlier detection:
 - Detect malformed or unusual tuples based on poor structural fit.

Why is this a Difficult Problem?

- ▶ We only have positive examples.
 - i.e., no information about what is not allowed.
- ▶ Naively OR-ing all strings gives a valid syntactic description.
 - However, it is completely useless, i.e., non-generalized.
- ▶ Syntactic description should be:
 - Compact,
 - Human-interpretable,
 - Fast to learn, and
 - Directly comparable across attributes.

Why not use Regular Expressions?

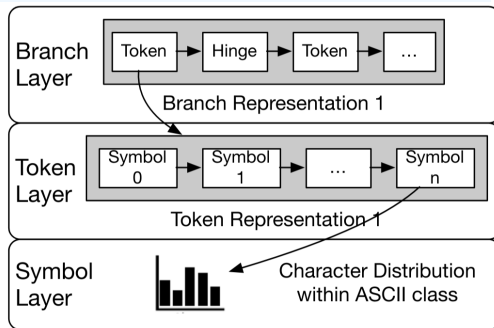
- ▶ Learning minimal regular expressions is NP-hard.
 - Existing algorithms are too slow.
 - Regular expressions are more expressive than needed.
- ▶ Real database columns often have:
 - Low variety of lengths.
 - Few structural variations.
 - Mostly finite languages.



XSTRUCTURE Architecture

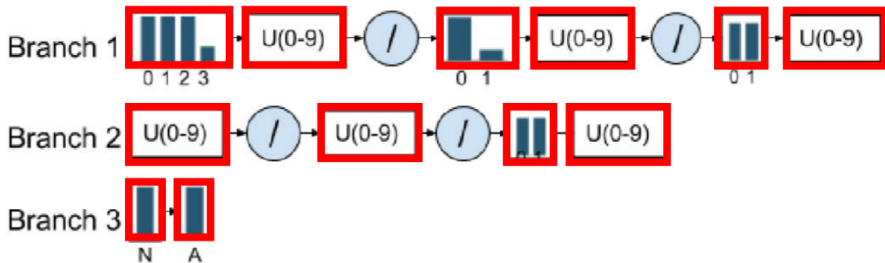
- ▶ Less expressive than regular expressions.
 - Represents a finite subset of regular expressions.
 - Sufficient for most database attributes.

- ▶ Three levels of representation:
 - Branch layer: alternative structural formats.
 - Token layer: sequences split by delimiters.
 - Symbol layer: distributions over characters.



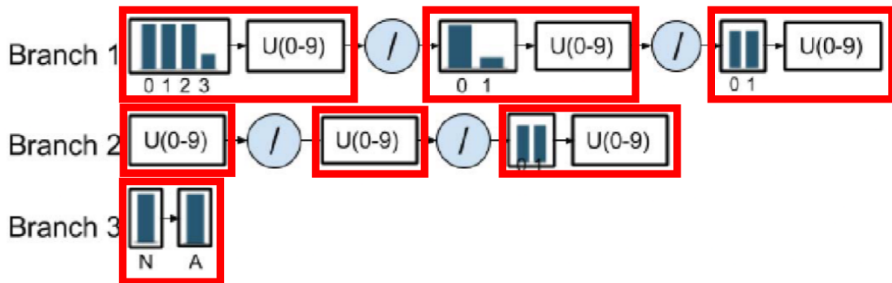
XSTRUCTURE Architecture – Symbol Layer

- ▶ The lowest layer, modeling each character position.
- ▶ Stores distribution over characters.
 - Encodes which characters appear.
 - Encodes frequencies for statistical generalization.



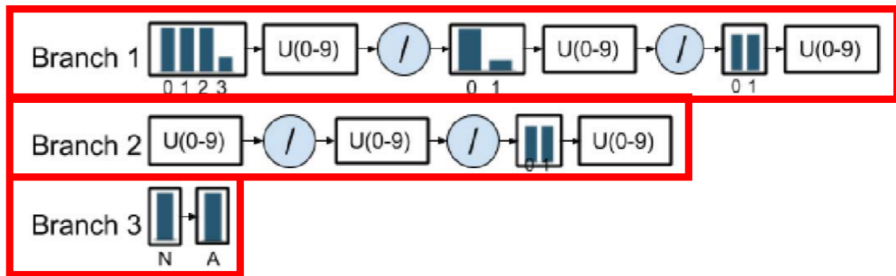
XSTRUCTURE Architecture – Token Layer

- ▶ Represents a fixed-length sequence of symbol layers.
- ▶ Tokens separated by delimiters (e.g., /, ., -).
- ▶ Example: 10/12/2018 tokenizes into 10, 12, 2018.



XSTRUCTURE Architecture – Branch Layer

- ▶ Top layer representing alternative syntactic formats.
 - Example: dates with 2-digit vs. 4-digit years.
- ▶ Each branch is a sequence of tokens plus delimiters.
- ▶ Only layer where variability in length is possible.



Learning Process

- ▶ Iterative, adding one tuple at a time:
 - Compute fit of new tuple with existing branches.
 - If fit above threshold, merge into best branch.
 - Otherwise, create a new branch.
- ▶ Repeat for all tuples in the column.

Learning Process – Branch and Merge

- ▶ Issue: branching threshold is unintuitive.
 - User instead specifies a maximum number of branches.
- ▶ Learning strategy:
 - Start with a low threshold.
 - If branch count exceeds maximum:
 - Merge two most similar branches.
 - Update threshold to the distance between them.
- ▶ Enables parallel learning:
 - Independently learned branches can be merged later.

Fit Scoring Explained

- ▶ Fit score measures how well a string matches a XSTRUCTURE.

- ▶ Defined at three levels:

→ Symbol-level: exact match or character class membership.

$$d(s_i, l_i) = \begin{cases} 0 & \text{if } s_i \in l_i \\ \alpha & \text{if } \text{class}(s_i) = \text{class}(l_i) \\ 1 & \text{otherwise} \end{cases}$$

→ Token-level: sum of symbol scores + length mismatch penalty.

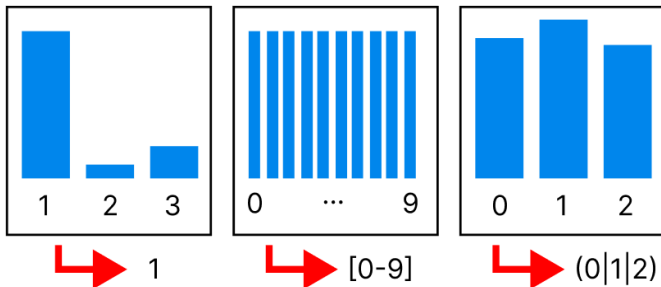
$$d(t_i, k_i) = \sum_{s_i \in t_i, l_i \in k_i} d(s_i, l_i) + |\text{len}(t_i) - \text{len}(k)|$$

→ Branch-level: minimum distance across branches.

$$d(t, X) = \min_{b \in B} \sum_{t_i \in t, k_i \in b} d(t_i, k_i)$$

Representation and Character Classes

- ▶ Symbol layers must be serialized for human readability.
- ▶ Use a χ^2 test to decide representation:
 - Single character (e.g., 1),
 - Standard character class (e.g., digit [0-9]),
 - OR-list of characters (e.g., (0|1|2)).



Comparing XSTRUCTURES

- ▶ Needed for label assignment and attribute comparison.

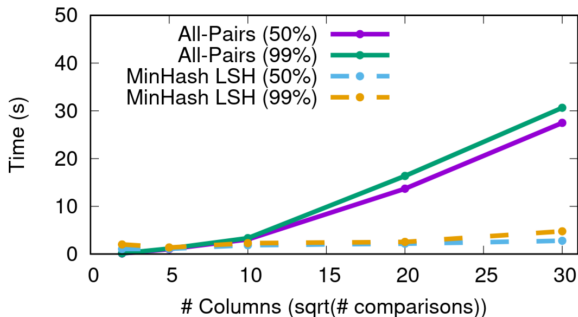
- ▶ Comparison yields two asymmetric scores:

- Fit of samples from A to B .
- Fit of samples from B to A .

- ▶ Estimation based on random sampling.

LSH for Approximate Similarity Search

- ▶ All-pairs comparisons are too expensive for large datasets.
- ▶ Generate triples $\langle \text{character}, \text{hinge}, \text{position} \rangle$ from XSTRUCTURES.
 - Example: $10/12 \rightarrow \langle 1, 0, 0 \rangle, \langle 0, 0, 1 \rangle, \langle 1, 1, 0 \rangle, \langle 2, 1, 1 \rangle$.
- ▶ Use MinHash to approximate Jaccard similarity across triple sets.



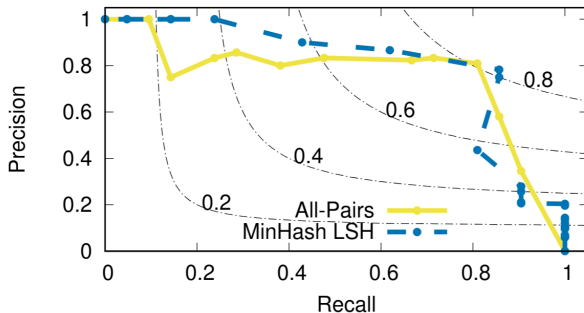
Evaluation – Automatic Label Assignment

- ▶ Match XSTRUCTURES to library of regex–label pairs.
- ▶ Compare assignments to manually curated ground truth.
- ▶ Achieves 94% accuracy on three datasets.

Dataset	total attrs.	correct matches	% matches
DWH	1504	1417	94%
CHE	307	294	95.5%
GOV	2476	2355	94.9%

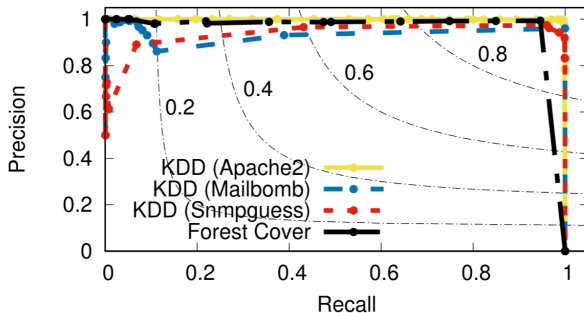
Evaluation – Attribute Similarity Search

- ▶ Task: find similar columns in datasets.
- ▶ Compared against manually labeled ground truth.
- ▶ Both all-pairs and LSH-based methods reach $\sim 80\%$ F1-score at best.



Evaluation – Syntax-Based Outlier Detection

- ▶ Fit XSTRUCTURE on subsets of data.
- ▶ Evaluate fit of new samples.
 - Outliers fit poorly into the learned structure.
- ▶ Achieves very high precision and recall in experiments.



Strengths

- ▶ Fast learning, even on large datasets.
- ▶ Human-readable outputs (regex-like).
- ▶ Supports probabilistic generation.
- ▶ Works with only positive examples.
- ▶ Parallelizable.

Limitations

- ▶ Assumes delimiter-based structure.
- ▶ Cannot represent cyclic languages.
- ▶ Cannot handle highly unstructured text.
- ▶ Hard to distinguish meaningful rare patterns from noise.
- ▶ Limited long-range dependencies and context-sensitive patterns.

Quiz

Visit **gosocrative.com** and enter room
name **BERNARD6902**

